

1. Consider an array containing the following 40 integers: (10 points)

5 2 4 4 0 1 6 7 3 1 1 0 5 1 5 4 4 5 7 0 6 1 0 7 5 2 7 6 5 3 7 0 5 5 7 1 1 2 6 5

Apply counting sort to sort this array and show all the steps of your work.

★ To apply counting sort, as its name implies, we need to count how many times each value appears in the array. This information is used fill in the array with the right number of copies of each value.

↳ only works well if elements in array are within a small range $0 \dots \text{max}$ of possible values (assuming we are sorting non-negative integers)

I) Traverse the array to find the maximum value max in the array $\Rightarrow \Theta(n)$

In the given array, the max is 7

II) Allocate an array of size $\text{max} + 1$ of counters, one for each possible value that can appear in the data array. All the counters are initialized to 0 $\Rightarrow \Theta(\text{max})$

	0	1	2	3	4	5	6	7
counters	0	0	0	0	0	0	0	0

III) Traverse the data array again, count how many times each value appears in the array, and record that information in the array of counters $\Rightarrow \Theta(n)$

	0	1	2	3	4	5	6	7
counters	5	7	3	2	4	9	4	6

Note that $5 + 7 + 3 + 2 + 4 + 9 + 4 + 6 = 40$

which is the length of the array.

IV) Traverse the array of counters and copy into the data array each value a number of times equal to the count specified for that value in the array of counters $\Rightarrow \Theta(\text{max})$ for traversing array + $\Theta(n)$ for writing

data: 0 0 0 0 0 1 1 1 1 1 1 2 2 2 3 3 4 4 4 4 5 5 5 5 5 5 5 5 6 6 6 6 7 7 7 7 7 7

	0	1	2	3	4	5	6	7
counters	0	0	0	0	0	0	0	0

the data array and decrementing all the counters multiple times each.

Array of counters can then be de-allocated

Final sorted Array:

0 0 0 0 0 1 1 1 1 1 1 2 2 2 3 3 4 4 4 4 5 5 5 5 5 5 5 5 6 6 6 6 7 7 7 7 7 7

2. Consider an array containing the following hexadecimal numbers: (10 points)

4EC1EEA9
520B6E78
1E90D74E
52DB6E42
5F05EF13
74284442
794E8117
55526E42

- RadixSort: to sort integers, we sort them first on their units digit (least significant digit) using a stable version of counting sort. Then we sort on their tens digit, then hundreds digit, etc.

Consider you are using a version of RadixSort that sorts on one byte at a time (two hexadecimal digits at a time). Under the stated circumstances, show all the steps when you apply radix sort on this array.

Sorting by two hexadecimal digits at a time

original array:

4EC1EEA9
520B6E78
1E90D74E
52DB6E42
5F05EF13
74284442
794E8117
55526E42

looking at last two digits

sort based on digits

I) sort array based on 2nd to last digit

5 F 0 5 E F 1 3 EF
7 9 4 E 8 1 1 7 81
1 E 9 0 D 7 4 E D7
5 2 D B 6 E 4 2 6E
7 4 2 8 4 4 4 2 44
5 5 5 2 6 E 4 2 6E
5 2 0 B 6 E 7 8 6E
4 E C 1 E E A 9 EE

stable algorithm
we break ties by sorting based on order it appears

Sort based on digits

II) sort array based on next two digits

7 4 2 8 4 4 4 2
5 2 D B 6 E 4 2
5 5 5 2 6 E 4 2
5 2 0 B 6 E 7 8
7 9 4 E 8 1 1 7
1 E 9 0 D 7 4 E
4 E C 1 E E A 9
5 F 0 5 E F 1 3

looking at next two digits

sort based on digits

28
DB
52
0B
4E
90
C1
05

III) sort array based on next two digits

5 F 0 5 E F 1 3 5F
5 2 0 B 6 E 7 8 52
7 4 2 8 4 4 4 2 74
7 9 4 E 8 1 1 7 79
5 5 5 2 6 E 4 2 55
1 E 9 0 0 7 4 E 1E
4 E C 1 E E A 9 4E
5 2 D B 6 E 4 2 52

sort

IV) Final sorted array:

1 E 9 0 D 7 4 E
4 E C 1 E E A 9
5 2 0 B 6 E 7 8
5 2 D B 6 E 4 2
5 5 5 2 6 E 4 2
5 F 0 5 E F 1 3
7 4 2 8 4 4 4 2
7 9 4 E 8 1 1 7

3. Construct a Huffman tree (variable-length encoding) for the following: (5 points)

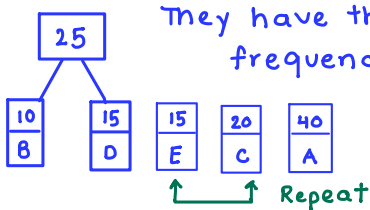
Symbol	A	B	C	D	E
Frequency	40	10	20	15	15

- Encode ABACABAD using the tree you generated. [1 point]
- Decode 100010111001010 using the tree you generated. [1 point]
- What compression gain (percent of improvement) do we get by using Huffman encoding (variable-length encoding) instead of a fixed-length encoding scheme. Draw the tree for the fixed-length encoding. (3 points)

10	15	15	20	40
B	D	E	C	A

First, we need to construct a Huffman tree by connecting Symbol / Frequency nodes

start by forming a tree to connect these, where the root is the sum of the frequencies
They have the smallest frequencies



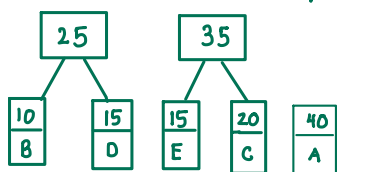
a) Encode ABACABAD using the three

we look at the path from the root to the leaf (or character)

$A = 0$
 $B = 100$
 $C = 111$
 $D = 101$
 $E = 110$

Using this, Encoding of ABACABAD is:

0100011101000101



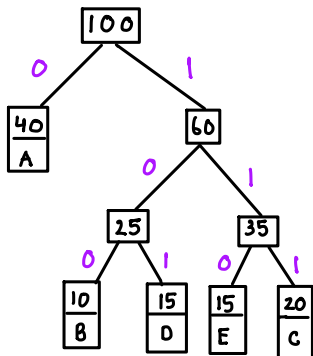
since these have smallest roots, connect!
or frequencies

b) Decoding 100010111001010

we look at: $A = 0$ and see what letters / characters correspond
 $B = 100$
 $C = 111$
 $D = 101$
 $E = 110$

1000 1011 1001 0101
B A D E A D A

Final Huffman Tree:



we assign 1 for right branches/edges and 0 for left branches/edges

c) To find compression gain:

For fixed length encoding,

Since we have 5 characters / symbols (A,B,C,D,E), we encode each with 3 bits.

$A = 000$
 $B = 001$
 $C = 010$
 $D = 011$
 $E = 100$

3 bits per character

There are other combinations that are wasted, like 111, etc.

Variable-length encoding:
We came up with:

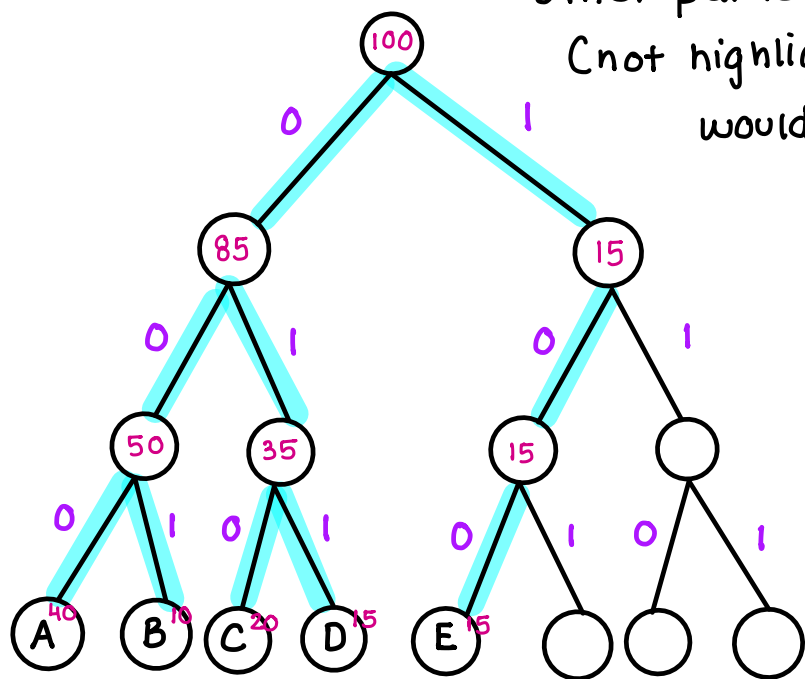
$A = 0$
 $B = 100$
 $C = 111$
 $D = 101$
 $E = 110$

avg bits for variable encoding

$= \sum \text{freq of } S \cdot \text{bits per } S$

BADEADA

tree using fixed length encoding



other paths
(not highlighted)
would be wasted.

$$\begin{aligned}
 &= \underbrace{(0.4)}_A \times 1 + \underbrace{(0.1)}_B \times 3 \\
 &+ \underbrace{(0.2)}_C \times 3 + \underbrace{(0.15)}_D \times 3 \\
 &+ \underbrace{(0.15)}_E \times 3 = \underline{2.2 \text{ bits}} \\
 &\quad \quad \quad \underline{\text{per}} \\
 &\quad \quad \quad \underline{\text{character}}
 \end{aligned}$$

$$\text{Compression rate} = \frac{f-v}{f} \times 100 \text{ where}$$

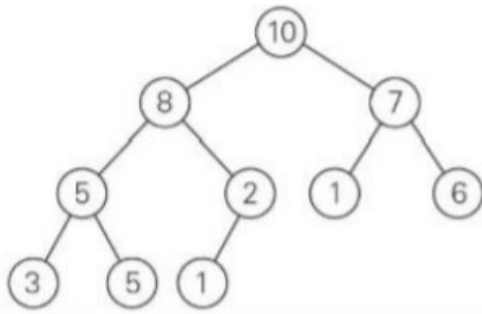
$$\begin{aligned}
 f: & \text{fixed} \\
 v: & \text{variable}
 \end{aligned}
 = \frac{3-2.2}{3} = \frac{0.8}{3} \times 100$$

$$= 0.2667 \times 100$$

$$= 26.26\%$$

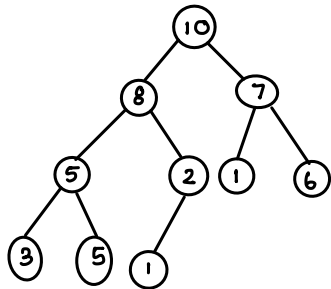
By using Huffman/ variable length encoding, we get a compression gain/ Improvement of 26.26%.

4. Consider the following binary tree. Print out the nodes in the following orders (10 points):



- Traverse the tree in preorder (2 points)
- Traverse the tree in inorder (2 points)
- Traverse the tree in postorder (2 points)
- How many internal nodes are there? (1 point)
- What is the maximum width of the tree? (1 point)
- What is the height of the tree? (1 point)
- What is the diameter of the tree? (1 point)

a) Preorder Traversal $\Rightarrow$ Visit root, then left Subtree, then right



10, 8, 5, 3, 5, 2, 1, 7, 1, 6

b) Inorder

Traversal $\Rightarrow$ First left Subtree, then root, then right subtree

3, 5, 5, 8, 1, 2, 10, 1, 7, 6

c) Postorder Traversal $\Rightarrow$ First left subtree, then right, then root

3, 5, 5, 1, 2, 8, 1, 6, 7, 10

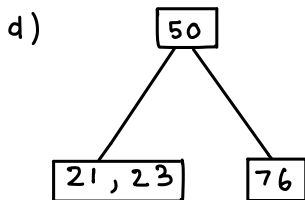
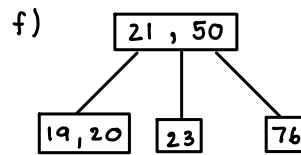
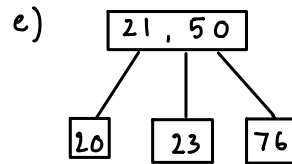
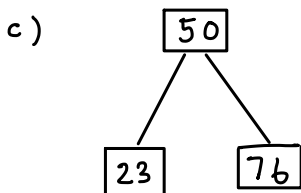
d) There are **5** internal nodes within this binary tree. An Internal node is a node which is not a leaf [in our case, 10, 8, 5, 2, and 7 are internal nodes].

e) The maximum width of the tree is the number of nodes that have the same depth. In our case, it is **4** since 5, 2, 1, and 6 have the same depth at level 3.

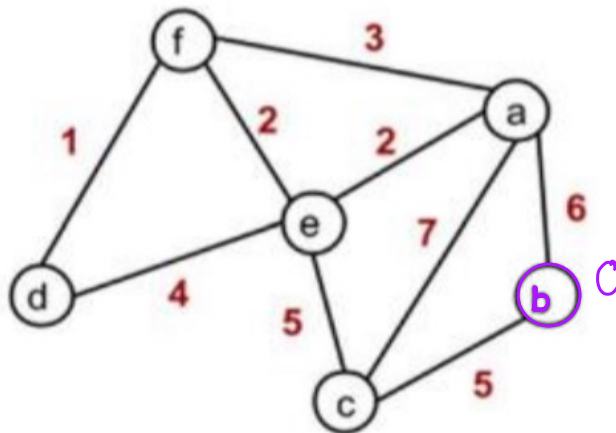
f) The height of the tree is the number of edges on the longest path from the root to a leaf. In our case, one of our longest paths is $10 \rightarrow 8 \rightarrow 5 \rightarrow 3$. so the height of the tree is **3**.

g) The diameter of the tree is the length of the longest path between any two nodes (not taking the direction of edges into account). A longest path: $3 \rightarrow 5 \rightarrow 8 \rightarrow 10 \rightarrow 7 \rightarrow 6$. Therefore the diameter is **5**.

5. Draw the 2-3 tree after inserting each of the following keys. Redraw the tree for each part (10 points)
- 50
 - 76
 - 23
 - 21
 - 20
 - 19



6. Apply Dijkstra's algorithm to compute the shortest paths starting from vertex **b** in the following graph. Show all the steps. (10 points):



Shortest paths

$b \rightarrow a = 6$
 $b \rightarrow c = 5$
 $b \rightarrow d = 10$
 $b \rightarrow e = 8$
 $b \rightarrow f = 9$
 $b \rightarrow b = 0$

- we start at $b(0)$, we initialize everything to infinity.
- Select $b(0)$ from priority queue
- look at possible paths from b and update
- pick $c(5)$ from priority queue
- update paths from c
- pick $a(6)$ from priority queue
- update paths from a
- pick $e(8)$ from priority queue
- update paths from e
- pick $f(9)$ from priority queue
- update paths from f
- update path to d since we have better one

Q:

a	b	c	d	e	f	
∞	0	∞	∞	∞	∞	$S = \{b\}$
6		5	∞	∞	∞	$S = \{b, c\}$
6			∞	10	∞	$S = \{b, c, a\}$
			∞	8	9	$S = \{b, c, a, e\}$
			12	9		$S = \{b, c, a, e, f\}$
			10			$S = \{b, c, a, e, f, d\}$